ColLab-FT iFogSim Simulation

Mode 1: Decentralized Fault-Tolerant Workflow Detailed Technical Analysis

Collaborative Fault-Tolerant Fog Computing

November 30, 2025

Contents

1 Overview **5**

2 System Configuration **5**
2.1 Key Parameters 5
2.2 Fault Distribution 5

3 Topology Architecture **5**
3.1 Fog Node Structure 5
3.2 Network Links 6

4 Task Generation and Arrival **6**
4.1 Task Profile Generation 6
4.2 Task Parameters 6
4.3 SLA Deadline Calculation 6
4.4 Example Calculation 7

5 Fault Injection and Prediction **7**
5.1 Fault Generation 7
5.2 Fault Type Selection 7
5.3 Fault Prediction Timeline 7
5.4 Fault Handling Workflow 7
5.4.1 At Prediction Time ($t_{predict}$) 7
5.4.2 At Fault Time (t_{fault}) 8
5.4.3 At Recovery Time ($t_{recover}$) 8
5.5 Realistic Fault Handling 8
5.5.1 Server Crash Behavior 8

6 Gossip Protocol **8**
6.1 Gossip Mechanism 8
6.1.1 Step 1: Snapshot Local Load 9
6.1.2 Step 2: Create Gossip Entry 9
6.1.3 Step 3: Send to Ring Neighbors 9

6.1.4 Step 4: Receive and Merge Updates . . . 9
6.2 Staleness Check . . . 9

7 Proactive Fault Prevention **9**
7.1 Critical Design Decision . . . 9
7.1.1 During Bid Evaluation . . . 9
7.1.2 During Local Server Selection . . . 10
7.2 Rationale . . . 10

8 Local Placement Decision **10**
8.1 Server Selection Algorithm . . . 10
8.1.1 Feasibility Check . . . 10
8.1.2 Predicted Fault Check . . . 10
8.1.3 Residual Score Calculation . . . 10
8.1.4 Selection . . . 11
8.2 Placement Outcomes . . . 11

9 Migration Workflow **11**
9.1 Phase 1: Intra-Fog Migration . . . 11
9.2 Phase 2: Inter-Fog Migration with Gossip-Based Scoring . . . 11
9.2.1 Step 1: Dynamic Weight Calculation . . . 11
9.2.2 Step 2: Fog Node Scoring Using Gossip Table . . . 12
9.2.3 Step 3: Candidate Selection . . . 12
9.2.4 Step 4: Bidding Request . . . 12

10 Bid Evaluation and Response **12**
10.1 Bid Calculation at Each Bidder Node . . . 12
10.1.1 Migration Time . . . 12
10.1.2 Resource Impact . . . 12
10.1.3 Bid Value . . . 12
10.1.4 Feasibility Check . . . 13
10.2 Bid Response . . . 13

11 Winner Selection **13**
11.1 Bid Collection and Filtering . . . 13
11.2 Effective Bid Calculation . . . 13
11.3 Headroom Calculation . . . 13
11.4 Winner Selection Criteria . . . 14
11.5 Fallback Strategy . . . 14

12 Container Migration Transfer **14**
12.1 Migration Start . . . 14
12.2 Container Placement at Destination . . . 14
12.3 Transfer Time Calculation . . . 14
12.4 Execution Scheduling . . . 15
12.5 Migration Finish . . . 15

13 Task Completion and SLA Settlement **15**
13.1 Completion Metrics . . . 15
13.2 Payment Settlement . . . 15

14 Trust-Based Reputation System **15**
14.1 Overview . . . 15
14.2 Trust Initialization . . . 16
14.3 Performance Metrics Evaluation . . . 16
14.3.1 Bandwidth Error . . . 16
14.3.2 Migration Time Error . . . 16
14.3.3 SLA Margin . . . 16
14.4 Dishonesty Detection . . . 17
14.4.1 Error Thresholds . . . 17
14.4.2 Dishonesty Flags . . . 17
14.5 Combined Trust Update . . . 17
14.6 Trust Update Equations . . . 18
14.6.1 Phase 1: Migration Performance (Multiplicative) . . . 18
14.6.2 Phase 2: SLA Performance (Additive) . . . 18
14.7 Trust-Based Bid Filtering . . . 19
14.7.1 Hard Threshold . . . 19
14.7.2 Soft Penalty (Effective Bid) . . . 19
14.8 Trust Recovery Mechanism . . . 19
14.8.1 Recovery After Honest Behavior . . . 19
14.9 Trust Metric Types . . . 20
14.9.1 Direct Verification (Bandwidth & Migration Time) . . . 20
14.9.2 Indirect Verification (SLA Performance) . . . 20
14.10 Gossip Load Verification . . . 20
14.10.1 Gossip Cannot Be Lied About . . . 20
14.10.2 SLA-Based Trust Detects Gossip Dishonesty . . . 20
14.11 Trust Configuration Parameters . . . 21
14.12 Trust Evolution Examples . . . 21
14.12.1 Example 1: Honest Node . . . 21
14.12.2 Example 2: Bandwidth Liar . . . 21
14.12.3 Example 3: SLA Violator . . . 21
14.12.4 Example 4: Combined Dishonesty . . . 22
14.13 Trust Impact on System Behavior . . . 22
14.13.1 Bid Competition . . . 22
14.13.2 Economic Incentive . . . 22
14.13.3 System-Wide Benefits . . . 22

15 Checkpointing and Progress Tracking **22**
15.1 Checkpointing . . . 22
15.2 Re-execution at Destination . . . 23

16 Formal Algorithms **23**
16.1 Algorithm 1: Fault Prediction and Injection . . . 23
16.2 Algorithm 2: Fault Prediction Handler . . . 23
16.3 Algorithm 3: Migration Decision with Fault Prevention . . . 24
16.4 Algorithm 4: Local Server Selection with Fault Avoidance . . . 25

16.5 Algorithm 5: Fault Occurrence Handler 26
16.6 Algorithm 6: Gossip Protocol for State Synchronization 27
16.7 Algorithm 7: Bid Generation and Cost Calculation 27

17 Mode 1 Results Summary **27**
17.1 Performance Metrics 28
17.2 Fault Tolerance 28
17.3 Migration Statistics 28
17.4 Migration Reasons 28
17.5 Network Overhead 29
17.6 Scheduling Performance 29
17.7 Economic Metrics 29

18 Edge Cases and Limitations **29**
18.1 Rare Post-Fault Migrations 29
18.1.1 Race Conditions 29
18.1.2 Empty Servers 30
18.2 Server Crash Impact 30

19 Key Advantages of Mode 1 **30**

20 Complete Fault Tolerance Workflow **31**
20.1 Detailed Workflow Steps 31
20.1.1 Step 1: Normal Operation 31
20.1.2 Step 2: Fault Prediction 31
20.1.3 Step 3: Migration Decision 31
20.1.4 Step 4: Container Transfer 32
20.1.5 Step 5: Actual Fault 32
20.1.6 Step 6: Recovery 32
20.1.7 Step 7: Continuous Monitoring 33

21 Mathematical Summary **33**
21.1 Key Equations 33
21.1.1 Load Calculation 33
21.1.2 Dynamic Weight Calculation 33
21.1.3 Fog Node Score 33
21.1.4 Bid Value 33
21.1.5 Winner Selection 33

22 Conclusion **33**
22.1 Key Implementation Insights 34

1 Overview

Mode 1 implements a **decentralized collaborative fault-tolerant fog computing system** with **gossip-based state sharing** for distributed decision-making. This mode enables fog nodes to make autonomous scheduling decisions based on shared resource state information, without centralized coordination.

2 System Configuration

2.1 Key Parameters

The system is configured with the following parameters from `collabft-config.yaml`:

- **Mode**: 1 (Decentralized with gossip)
- **Simulation Duration**: 3600 seconds (60 minutes)
- **Gossip Interval**: 7 seconds
- **Fault Prediction Lead Time**: 60 seconds
- **Mean Time Between Failures (MTBF)**: 60 seconds
- **Recovery Time**: 240 seconds
- **Topology**: 10 fog nodes with 3-4 servers each, 6 edge devices per fog node

2.2 Fault Distribution

Faults are distributed according to the following probabilities:

$$P_{cpu} = 0.33, \quad P_{bandwidth} = 0.34, \quad P_{crash} = 0.33 \tag{1}$$

3 Topology Architecture

3.1 Fog Node Structure

The system consists of:

- **10 fog nodes** arranged in a ring topology for gossip propagation
- Each fog node has **3-4 servers**
- Each server has the following capacity:
  - CPU: 2000-2200 MIPS
  - RAM: 4096 MB
  - Bandwidth: 3500-4500 Mbps
  - Storage: 100 GB

3.2 Network Links

Network latencies and bandwidth characteristics:

$$L_{fog} = 8 \text{ ms (inter-fog latency)} \tag{2}$$
$$B_{fog} = 1500 \text{ Mbps (inter-fog bandwidth)} \tag{3}$$
$$L_{cloud} = 70 \text{ ms (fog-to-cloud latency)} \tag{4}$$
$$B_{cloud} = 150 \text{ Mbps (fog-to-cloud bandwidth)} \tag{5}$$
$$L_{edge} = 3 \text{ ms (edge-to-fog latency)} \tag{6}$$
$$B_{edge} = 1500 \text{ Mbps (edge-to-fog bandwidth)} \tag{7}$$

4 Task Generation and Arrival

4.1 Task Profile Generation

Each edge device generates tasks with exponential inter-arrival times:

$$\text{InterArrival} = -\bar{\tau} \cdot \ln(1 - U), \quad U \sim \text{Uniform}(0, 1), \quad \bar{\tau} = 160 \text{ seconds} \tag{8}$$

4.2 Task Parameters

Task parameters are sampled from uniform distributions:

$$R_{cpu} \sim \text{Uniform}[200, 600] \text{ MIPS} \tag{9}$$
$$T_{exec} \sim \text{Uniform}[200, 400] \text{ seconds} \tag{10}$$
$$R_{mem} \sim \text{Uniform}[128, 512] \text{ MB} \tag{11}$$
$$R_{bw} \sim \text{Uniform}[150, 550] \text{ Mbps} \tag{12}$$
$$S_{container} \sim \text{Uniform}[200, 420] \text{ MB} \tag{13}$$

4.3 SLA Deadline Calculation

The SLA deadline for each task is calculated as:

$$D_{sla} = T_{arrival} + T_{exec} + T_{net} + T_{slack} + T_{mig} \tag{14}$$

where:

$$T_{exec} = \text{Runtime}_{\text{task}} \text{ (base execution time)} \tag{15}$$
$$T_{net} = 0.04 \times T_{exec} \text{ (network overhead ratio)} \tag{16}$$
$$T_{slack} = 0.06 \times T_{exec} \text{ (slack ratio for flexibility)} \tag{17}$$
$$T_{mig} = t_{pause} + \frac{S_{container}}{B_{inter}/8} + t_{resume} \tag{18}$$

With migration pause/resume times:

$$t_{pause} = 1.5 \text{ seconds} \tag{19}$$
$$t_{resume} = 1.5 \text{ seconds} \tag{20}$$
$$B_{inter} = 1500 \text{ Mbps (inter-fog bandwidth)} \tag{21}$$

4.4 Example Calculation

For $T_{exec} = 260$ seconds and $S_{container} = 240$ MB:

$$T_{net} = 0.04 \times 260 = 10.4 \text{ s} \tag{22}$$

$$T_{slack} = 0.06 \times 260 = 15.6 \text{ s} \tag{23}$$

$$T_{mig} = 1.5 + \frac{240}{1500/8} + 1.5 = 1.5 + 1.28 + 1.5 = 4.28 \text{ s} \tag{24}$$

$$D_{sla} = T_{arrival} + 260 + 10.4 + 15.6 + 4.28 = T_{arrival} + 290.28 \text{ s} \tag{25}$$

5 Fault Injection and Prediction

5.1 Fault Generation

The `FaultInjector` generates faults following a Poisson process with exponential inter-arrival times:

$$\text{TimeBetweenFaults} = -\text{MTBF} \cdot \ln(1 - U), \quad \text{MTBF} = 45 \text{ seconds} \tag{26}$$

5.2 Fault Type Selection

Fault types are randomly selected based on probabilities:

$$\text{FaultType} = \begin{cases} \text{CPU FAILURE} & \text{if } U < 0.33 \text{ (CPU reduced to 20\%)} \\ \text{BANDWIDTH DEGRADATION} & \text{if } 0.33 \leq U < 0.67 \text{ (BW reduced to 30\%)} \\ \text{SERVER CRASH} & \text{if } 0.67 \leq U < 1.0 \text{ (complete failure)} \end{cases} \tag{27}$$

5.3 Fault Prediction Timeline

For each fault at time t_{fault}:

1. **Prediction Event** at $t_{predict} = t_{fault} - 60$ seconds
2. **Actual Fault Event** at t_{fault}
3. **Recovery Event** at $t_{recover} = t_{fault} + 240$ seconds

5.4 Fault Handling Workflow

5.4.1 At Prediction Time ($t_{predict}$)

1. Server marked with predictedFailureAt $= t_{fault}$
2. For each container on the server:
   - Checkpoint progress: remaining_work = remaining_work $- (t_{now} - t_{start}) \times$ host_share_mips
   - Remove container from server
   - Trigger migration with reason: `fault_predicted_<type>`

5.4.2 At Fault Time (t_{fault})

1. Apply fault degradation:

$$\text{degradation} = \begin{cases} \text{cpuFactor} = 0.2 & \text{for CPU FAILURE} \\ \text{bwFactor} = 0.3 & \text{for BANDWIDTH DEGRADATION} \\ \text{failed} = \text{true} & \text{for SERVER CRASH} \end{cases} \tag{28}$$

2. For remaining containers (if any):
   - Checkpoint and remove
   - Trigger migration with reason: `fault_<type>`

5.4.3 At Recovery Time ($t_{recover}$)

Restore server state:

$$\text{failed} = \text{false} \tag{29}$$
$$\text{cpuFactor} = 1.0 \tag{30}$$
$$\text{bwFactor} = 1.0 \tag{31}$$
$$\text{predictedFailureAt} = -1 \tag{32}$$

5.5 Realistic Fault Handling

5.5.1 Server Crash Behavior

In real-world systems, crashed servers are **inaccessible**. The simulation implements realistic behavior:

For `SERVER_CRASH` faults:

- Containers on crashed server are **lost** (cannot checkpoint)
- All progress is discarded: $W_{remaining} = W_{total}$
- Tasks restart from beginning on new server
- This reflects reality: crashed servers = lost work

For `CPU_FAILURE` and `BANDWIDTH_DEGRADATION`:

- Server remains accessible (just degraded)
- Containers can be checkpointed normally
- Progress is preserved: $W_{remaining}$ maintained
- Migration proceeds with partial work

6 Gossip Protocol

6.1 Gossip Mechanism

Every 7 seconds, the `GossipAgent` executes the following protocol for each fog node i:

6.1.1 Step 1: Snapshot Local Load

$$L^i_{cpu} = \frac{\sum_{s \in \text{servers}_i} \text{usedCpu}_s}{\sum_{s \in \text{servers}_i} \text{capacity}^s_{cpu} \times \text{factor}^s_{cpu}} \tag{33}$$

$$L^i_{mem} = \frac{\sum_{s \in \text{servers}_i} \text{usedRam}_s}{\sum_{s \in \text{servers}_i} \text{capacity}^s_{mem}} \tag{34}$$

$$L^i_{bw} = \frac{\sum_{s \in \text{servers}_i} \text{usedBw}_s}{\sum_{s \in \text{servers}_i} \text{capacity}^s_{bw} \times \text{factor}^s_{bw}} \tag{35}$$

6.1.2 Step 2: Create Gossip Entry

Create a gossip state entry:

$$\text{GossipStateEntry}(L^i_{cpu}, L^i_{mem}, L^i_{bw}, t_{timestamp}) \tag{36}$$

6.1.3 Step 3: Send to Ring Neighbors

Each node sends its full state table (including entries from other nodes) to its next neighbor in the ring topology.

6.1.4 Step 4: Receive and Merge Updates

Upon receiving gossip, merge entries into local state table.

6.2 Staleness Check

An entry at timestamp t_{entry} is considered **stale** at time t_{now} if:

$$t_{now} - t_{entry} > 7 \times 3 = 21 \text{ seconds} \tag{37}$$

Stale entries are not used for scoring during migration decisions but remain in the table.

7 Proactive Fault Prevention

7.1 Critical Design Decision

To ensure **proactive fault tolerance**, servers with predicted faults are **completely excluded** from accepting new containers:

7.1.1 During Bid Evaluation

When responding to migration requests:

$$\text{hasPredictedFault} = \exists s \in \text{servers} : s.\text{isPredictedToFail}() \tag{38}$$

$$\text{feasible} = \neg\text{hasPredictedFault} \wedge \text{resourcesFeasible} \wedge \text{deadlineFeasible} \tag{39}$$

If **any** server in the fog node has a predicted fault, the entire node rejects all migration bids.

7.1.2 During Local Server Selection

When placing containers locally:

$$S_{eligible} = \{s \in \text{servers} : \neg s.\text{isPredictedToFail}() \wedge s.\text{residualScore}(profile) > 0\} \quad (40)$$

Servers with predicted faults are skipped entirely, preventing any new placements.

7.2 Rationale

This strict policy ensures:

- Containers migrate **before** faults occur, not after
- No containers arrive between prediction and actual fault
- Minimizes `fault_*` migrations (only `fault_predicted_*` should occur)
- Realistic proactive fault tolerance behavior

8 Local Placement Decision

When a task arrives at fog node i:

8.1 Server Selection Algorithm

8.1.1 Feasibility Check

For each server s, determine feasibility:

$$\text{Feasible}(s) = \begin{cases} \text{true} & \text{if } \frac{\text{usedCpu}_s + R_{cpu}}{\text{cap}^s_{cpu} \times f^s_{cpu}} \leq 1.0 \\ & \text{and } \frac{\text{usedMem}_s + R_{mem}}{\text{cap}^s_{mem}} \leq 1.0 \\ & \text{and } \frac{\text{usedBw}_s + R_{bw}}{\text{cap}^s_{bw} \times f^s_{bw}} \leq 1.0 \\ \text{false} & \text{otherwise} \end{cases} \quad (41)$$

8.1.2 Predicted Fault Check

Servers with predicted faults are **excluded**:

$$\text{Eligible}(s) = \neg s.\text{isPredictedToFail}() \quad (42)$$

8.1.3 Residual Score Calculation

For eligible servers, if feasible, compute the residual score:

$$\begin{aligned} \text{ResidualScore}(s) = &\left(1 - \frac{\text{usedCpu}_s + R_{cpu}}{\text{cap}^s_{cpu} \times f^s_{cpu}}\right) \\ &+ \left(1 - \frac{\text{usedMem}_s + R_{mem}}{\text{cap}^s_{mem}}\right) \\ &+ \left(1 - \frac{\text{usedBw}_s + R_{bw}}{\text{cap}^s_{bw} \times f^s_{bw}}\right) \end{aligned} \quad (43)$$

8.1.4 Selection

Select the server with the highest ResidualScore > 0 among eligible (non-predicted-to-fail) servers.

8.2 Placement Outcomes

- **Local Placement Success**: Container placed on selected server, schedule completion event, record placement metrics
- **Local Placement Failure**: Trigger migration workflow with reason `local_infeasible` or `local_infeasible_fault`

9 Migration Workflow

9.1 Phase 1: Intra-Fog Migration

Attempt to place on another server within the same fog node:

1. For each local server: Calculate residualScore
2. If any server has score > 0:
   - Place container locally
   - Record INTRA_FOG migration
   - Schedule completion
   - DONE
3. Else: Proceed to Phase 2

9.2 Phase 2: Inter-Fog Migration with Gossip-Based Scoring

9.2.1 Step 1: Dynamic Weight Calculation

Compute urgency-aware weights for the container:

$$\text{total} = R_{cpu} + R_{mem} + R_{bw} \tag{44}$$

$$p_{cpu} = \frac{R_{cpu}}{\text{total}}, \quad p_{mem} = \frac{R_{mem}}{\text{total}}, \quad p_{bw} = \frac{R_{bw}}{\text{total}} \tag{45}$$

$$\text{slack} = \max(10^{-3}, D_{sla} - t_{now}) \tag{46}$$

$$U = \frac{1}{\text{slack}}, \quad U_{norm} = \min(1.0, U \times 0.1) \tag{47}$$

Bandwidth weight adjustment (increases with urgency):

$$\gamma = p_{bw} + U_{norm} \times (1 - p_{bw}) \tag{48}$$

Remaining weight distribution:

$$W_{remaining} = 1 - \gamma \tag{49}$$

$$\alpha = W_{remaining} \times \frac{p_{cpu}}{p_{cpu} + p_{mem}} \tag{50}$$

$$\beta = W_{remaining} \times \frac{p_{mem}}{p_{cpu} + p_{mem}} \tag{51}$$

Verify normalization: $\alpha + \beta + \gamma = 1$

9.2.2 Step 2: Fog Node Scoring Using Gossip Table

For each fog node j in the gossip state table (excluding self and stale entries):

$$\text{Score}(j) = \alpha \times (1 - L^j_{cpu}) + \beta \times (1 - L^j_{mem}) + \gamma \times (1 - L^j_{bw}) \tag{52}$$

9.2.3 Step 3: Candidate Selection

1. Sort nodes by Score(j) in descending order
2. Take top min(maxFogBidders, available_nodes) candidates
3. If trust enabled: filter out nodes with trust < 0.5
4. Always include cloud if cloudId ≥ 0

9.2.4 Step 4: Bidding Request

Send `BID_REQUEST` to all selected candidates with:

```
MigrationRequest(container, originId)
```

10 Bid Evaluation and Response

10.1 Bid Calculation at Each Bidder Node

10.1.1 Migration Time

$$T_{mig} = t_{pause} + \frac{S_{container}}{B_{claimed}/8} + t_{resume} \tag{53}$$

where $B_{claimed}$ is the minimum of uplink bandwidth and inter-fog link bandwidth.

10.1.2 Resource Impact

$$\text{Impact} = \frac{R_{cpu}}{\text{cap}_{cpu}} + \frac{R_{mem}}{\text{cap}_{mem}} + \frac{R_{bw}}{B_{claimed}} \tag{54}$$

10.1.3 Bid Value

$$\text{BidValue} = T_{mig} + k \times \text{Impact}, \quad k = 0.6 \tag{55}$$

10.1.4 Feasibility Check

Critical: Check for predicted faults first:

$$\text{hasPredictedFault} = \exists s \in \text{servers} : s.\text{isPredictedToFail}() \tag{56}$$

$$\text{Feasible} = \begin{cases} \text{true} & \text{if } \neg\text{hasPredictedFault} \\ & \text{and } \frac{\text{usedCpu}+R_{cpu}}{\text{cap}_{cpu}} \leq 1.0 \\ & \text{and } \frac{\text{usedMem}+R_{mem}}{\text{cap}_{mem}} \leq 1.0 \\ & \text{and } \frac{\text{usedBw}+R_{bw}}{\text{cap}_{bw}} \leq 1.0 \\ & \text{and } T_{mig} \leq (D_{sla} - t_{now}) \\ \text{false} & \text{otherwise} \end{cases} \tag{57}$$

Note: If any server has a predicted fault, the bid is immediately marked infeasible, regardless of resources or timing.

10.2 Bid Response

Send bid response back to origin:

```
BidResponse(bidderId, containerId, feasible,
    bidValue, claimedBw, claimedMigTime)
```

11 Winner Selection

11.1 Bid Collection and Filtering

At the origin fog node, collect all bid responses and:

1. **Trust Check** (disabled by default in Mode 1):

$$\text{Reject if trustEnabled} \wedge \text{trust(bidderId)} < 0.5 \tag{58}$$

2. **Re-evaluate with Local Context**:
   - Recalculate projected loads
   - Verify feasibility with local knowledge
   - Compute effective bid

11.2 Effective Bid Calculation

$$\text{EffectiveBid} = \begin{cases} \frac{\text{BidValue}}{\max(10^{-6}, \text{trust(bidderId)})} & \text{if trustEnabled} \\ \text{BidValue} & \text{otherwise} \end{cases} \tag{59}$$

11.3 Headroom Calculation

Remaining capacity (headroom):

$$\text{Headroom} = (1 - L_{cpu}^{proj}) + (1 - L_{mem}^{proj}) + (1 - L_{bw}^{proj}) \tag{60}$$

11.4 Winner Selection Criteria

Lexicographic ordering:

$$\text{Winner} = \arg\min_{\text{feasible}} \{\text{EffectiveBid}, -\text{Headroom}, \text{Latency}\} \tag{61}$$

1. **Primary**: Lowest EffectiveBid among feasible bids
2. **Tiebreaker 1**: Highest Headroom
3. **Tiebreaker 2**: Lowest Latency

11.5 Fallback Strategy

- If no feasible fog bids: **Cloud** (always accepts as last resort)
- If cloud also infeasible: **Enqueue in pending queue** for retry

12 Container Migration Transfer

12.1 Migration Start

Upon winner selection, send `MIGRATION_START` to winner with:

- Container profile
- Origin ID
- Source host name
- Migration kind (INTRA_FOG, INTER_FOG, CLOUD)
- Link bandwidth
- Link latency

12.2 Container Placement at Destination

1. Select local server using residualScore
2. Add container to server
3. Update resource usage

12.3 Transfer Time Calculation

With N active transfers on the link:

$$B_{effective} = \frac{B_{link}}{N} \tag{62}$$

$$T_{transfer} = \frac{S_{container}}{B_{effective}/8} + L_{latency} \tag{63}$$

12.4 Execution Scheduling

$$T_{exec} = \frac{\text{RemainingWork}_{\text{MI}}}{\text{HostShare}_{\text{MIPS}}} \tag{64}$$

where:

$$\text{HostShare}_{\text{MIPS}} = \frac{\text{ServerCPU} \times f_{cpu}}{\text{NumContainers on server}} \tag{65}$$

$$T_{complete} = t_{now} + T_{transfer} + T_{exec} \tag{66}$$

Schedule `CompletionNotice` event at $T_{complete}$.

12.5 Migration Finish

Send `MIGRATION_FINISH` back to origin for payment settlement.

13 Task Completion and SLA Settlement

13.1 Completion Metrics

$$T_{makespan} = T_{complete} - T_{arrival} \tag{67}$$

$$\text{SLA}_{\text{met}} = (T_{complete} \leq D_{sla}) \tag{68}$$

$$\text{Violation}_{\text{ratio}} = \frac{\text{violations}}{\text{total tasks}} \tag{69}$$

13.2 Payment Settlement

If migrated to different node:

- **Payment Amount** = BidValue (from winning bid)
- **Transfer**:
  - Debit: Owner fog node
  - Credit: Winner fog node

14 Trust-Based Reputation System

14.1 Overview

The trust mechanism evaluates fog node performance across three dimensions to detect and penalize dishonest behavior:

1. **Bandwidth Claims**: Actual vs claimed migration bandwidth
2. **Migration Time**: Actual vs claimed migration duration
3. **SLA Performance**: Whether tasks complete before deadline (indicates CPU/memory allocation honesty)

14.2 Trust Initialization

All fog nodes start with maximum trust:

$$\text{trust}(i) = 1.0, \quad \forall i \in \text{FogNodes} \tag{70}$$

Trust values are bounded:

$$\text{trust}(i) \in [0, 1], \quad \forall i \tag{71}$$

14.3 Performance Metrics Evaluation

14.3.1 Bandwidth Error

During migration, actual bandwidth is measured:

$$B_{actual} = \frac{S_{container}}{T_{transfer}} \times 8 \text{ (Mbps)} \tag{72}$$

Relative bandwidth error:

$$e_{bw} = \frac{|B_{claimed} - B_{actual}|}{\max(10^{-6}, B_{claimed})} \tag{73}$$

14.3.2 Migration Time Error

Actual migration time includes pause, transfer, and resume:

$$T_{mig,actual} = t_{pause} + T_{transfer} + t_{resume} \tag{74}$$

Relative migration time error:

$$e_{mig} = \frac{|T_{mig,claimed} - T_{mig,actual}|}{\max(10^{-6}, T_{mig,claimed})} \tag{75}$$

14.3.3 SLA Margin

Deadline margin indicates execution performance:

$$M_{sla} = D_{sla} - T_{complete} \tag{76}$$

Normalized margin (relative to deadline):

$$m_{sla} = \frac{M_{sla}}{\max(10^{-6}, D_{sla})} \tag{77}$$

where:

$$m_{sla} = \begin{cases} > 0 & \text{SLA met with margin} \\ = 0 & \text{SLA exactly met} \\ < 0 & \text{SLA violated} \end{cases} \tag{78}$$

14.4 Dishonesty Detection

14.4.1 Error Thresholds

Performance metrics are compared against thresholds:

$$\tau_{bw} = 0.3 \text{ (30\% bandwidth error allowed)} \tag{79}$$

$$\tau_{mig} = 0.5 \text{ (50\% migration time error allowed)} \tag{80}$$

$$\tau_{sla} = 0.1 \text{ (10\% deadline violation allowed)} \tag{81}$$

14.4.2 Dishonesty Flags

$$d_{bw} = \begin{cases} \text{true} & \text{if } e_{bw} > \tau_{bw} \\ \text{false} & \text{otherwise} \end{cases} \tag{82}$$

$$d_{mig} = \begin{cases} \text{true} & \text{if } e_{mig} > \tau_{mig} \\ \text{false} & \text{otherwise} \end{cases} \tag{83}$$

$$d_{sla} = \begin{cases} \text{true} & \text{if } m_{sla} < -\tau_{sla} \\ \text{false} & \text{otherwise} \end{cases} \tag{84}$$

14.5 Combined Trust Update

Trust is updated once per task completion, incorporating all three metrics:

Algorithm 1 Combined Trust Evaluation

Require: e_{bw}, e_{mig}, m_{sla}: Performance metrics
Require: τ_{bw}, τ_{mig}, τ_{sla}: Error thresholds
Require: $\delta_{decay} = 0.5$: Decay factor
Require: $\beta_{success} = 0.02$: SLA success bonus
Require: $\beta_{violation} = 0.10$: SLA violation penalty
Ensure: Updated trust value

$d_{bw} \leftarrow (e_{bw} > \tau_{bw})$
$d_{mig} \leftarrow (e_{mig} > \tau_{mig})$
$d_{sla} \leftarrow (m_{sla} < -\tau_{sla})$
$t_{current} \leftarrow \text{trust}(i)$
$t_{updated} \leftarrow t_{current}$

if d_{bw} **or** d_{mig} **then** ▷ Multiplicative decay for explicit lies
    $t_{updated} \leftarrow t_{current} \times \delta_{decay}$
end if

if d_{sla} **then** ▷ Additive penalty for SLA violations
    $t_{updated} \leftarrow \max(0, t_{updated} - \beta_{violation})$
else if $\neg d_{bw}$ **and** $\neg d_{mig}$ **then** ▷ Bonus only if no lies
    $t_{updated} \leftarrow \min(1, t_{updated} + \beta_{success})$
end if

$\text{trust}(i) \leftarrow t_{updated}$
return $t_{updated}$

14.6 Trust Update Equations

14.6.1 Phase 1: Migration Performance (Multiplicative)

If bandwidth or migration time errors exceed thresholds:

$$t'(i) = \begin{cases} t(i) \times \delta_{decay} & \text{if } d_{bw} \vee d_{mig} \\ t(i) & \text{otherwise} \end{cases} \tag{85}$$

where $\delta_{decay} = 0.5$ (50% penalty for dishonest claims).

14.6.2 Phase 2: SLA Performance (Additive)

Applied after migration performance evaluation:

$$t''(i) = \begin{cases} \max(0, t'(i) - \beta_{violation}) & \text{if } d_{sla} \\ \min(1, t'(i) + \beta_{success}) & \text{if } \neg d_{bw} \wedge \neg d_{mig} \wedge \neg d_{sla} \\ t'(i) & \text{otherwise} \end{cases} \tag{86}$$

where:

$$\beta_{violation} = 0.10 \text{ (10\% penalty for SLA violations)} \tag{87}$$

$$\beta_{success} = 0.02 \text{ (2\% reward for good performance)} \tag{88}$$

14.7 Trust-Based Bid Filtering

14.7.1 Hard Threshold

Fog nodes below the trust threshold are excluded from bidding:

$$\theta_{trust} = 0.5 \tag{89}$$

$$\text{Eligible}(i) = \begin{cases} \text{true} & \text{if trust}(i) \geq \theta_{trust} \\ \text{false} & \text{otherwise} \end{cases} \tag{90}$$

14.7.2 Soft Penalty (Effective Bid)

Low-trust nodes have inflated bid costs:

$$\text{EffectiveBid}(i) = \frac{\text{BidValue}(i)}{\max(10^{-6}, \text{trust}(i))} \tag{91}$$

Example:

- BidValue = 10.0, trust = 1.0 $\rightarrow$ EffectiveBid = 10.0
- BidValue = 10.0, trust = 0.5 $\rightarrow$ EffectiveBid = 20.0
- BidValue = 10.0, trust = 0.25 $\rightarrow$ EffectiveBid = 40.0

14.8 Trust Recovery Mechanism

14.8.1 Recovery After Honest Behavior

The additive recovery ensures trust can increase over time:

$$t_{recovered} = \min(1.0, t_{current} + \beta_{success}) \tag{92}$$

Recovery Time Analysis:
Starting from minimum trust after dishonest behavior:

$$t_0 = 0.5 \times 0.5 = 0.25 \text{ (after two violations)} \tag{93}$$

Recovery trajectory with consistent honest behavior ($\beta_{success} = 0.02$ per task):

$$t_1 = 0.25 + 0.02 = 0.27 \tag{94}$$
$$t_5 = 0.25 + 5 \times 0.02 = 0.35 \tag{95}$$
$$t_{10} = 0.25 + 10 \times 0.02 = 0.45 \tag{96}$$
$$t_{15} = 0.25 + 15 \times 0.02 = 0.55 \text{ (crosses threshold)} \tag{97}$$

Recovery to threshold: 15 honest tasks required.

14.9 Trust Metric Types

14.9.1 Direct Verification (Bandwidth & Migration Time)

These metrics are **directly measured** during migration:

- Fog nodes cannot lie about resources used in migration
- Origin node observes actual transfer time and bandwidth
- Errors indicate network variability or dishonest claiming

14.9.2 Indirect Verification (SLA Performance)

SLA violations indirectly indicate:

- **CPU under-allocation**: Container gets less CPU than expected
- **Memory under-allocation**: Container experiences swapping/paging
- **Bandwidth under-allocation**: Network I/O slower than claimed
- **Resource contention**: Fog node accepted too many containers

This catches execution-time dishonesty that cannot be verified directly.

14.10 Gossip Load Verification

14.10.1 Gossip Cannot Be Lied About

The gossip protocol broadcasts **measured** load values:

$$L^i_{resource} = \frac{\sum_{s \in \text{servers}_i} \text{measured\_usage}_s}{\sum_{s \in \text{servers}_i} \text{capacity}_s} \tag{98}$$

These are computed from actual server state, not claimed values. Fog nodes cannot artificially report lower loads.

14.10.2 SLA-Based Trust Detects Gossip Dishonesty

If a fog node could lie via gossip:

1. Reports low CPU load to attract containers
2. Actually has high load (CPU oversubscribed)
3. Containers execute slowly
4. SLA violations occur
5. Trust decays via d_{sla} flag

Thus, SLA-based trust indirectly verifies gossip honesty.

Parameter	Value	Description
δ_{decay}	0.5	Multiplicative decay factor
$\beta_{success}$	0.02	Additive reward for SLA success
$\beta_{violation}$	0.10	Additive penalty for SLA violation
θ_{trust}	0.5	Minimum trust for bidding
τ_{bw}	0.3	Bandwidth error threshold (30%)
τ_{mig}	0.5	Migration time error threshold (50%)
τ_{sla}	0.1	SLA margin threshold (10%)

Table 1: Trust System Configuration

14.11 Trust Configuration Parameters

14.12 Trust Evolution Examples

14.12.1 Example 1: Honest Node

Initial: $t_0 = 1.0$

Task 1: $e_{bw} = 0.05$, $e_{mig} = 0.10$, $m_{sla} = 0.15$ (all thresholds passed)

$$d_{bw} = \text{false}, \quad d_{mig} = \text{false}, \quad d_{sla} = \text{false} \tag{99}$$

$$t_1 = \min(1.0, 1.0 + 0.02) = 1.0 \tag{100}$$

Remains at maximum trust.

14.12.2 Example 2: Bandwidth Liar

Initial: $t_0 = 1.0$

Task 1: $e_{bw} = 0.40$ (exceeds $\tau_{bw} = 0.3$), $e_{mig} = 0.10$, $m_{sla} = 0.05$

$$d_{bw} = \text{true} \tag{101}$$

$$t_1 = 1.0 \times 0.5 = 0.5 \tag{102}$$

Task 2: Honest behavior, $e_{bw} = 0.10$, $e_{mig} = 0.15$, $m_{sla} = 0.08$

$$d_{bw} = \text{false}, \quad d_{mig} = \text{false}, \quad d_{sla} = \text{false} \tag{103}$$

$$t_2 = \min(1.0, 0.5 + 0.02) = 0.52 \tag{104}$$

Slowly recovers with honest behavior.

14.12.3 Example 3: SLA Violator

Initial: $t_0 = 1.0$

Task 1: $e_{bw} = 0.10$, $e_{mig} = 0.20$, $m_{sla} = -0.15$ (violated by 15%)

$$d_{bw} = \text{false}, \quad d_{mig} = \text{false}, \quad d_{sla} = \text{true} \tag{105}$$

$$t_1 = \max(0, 1.0 - 0.10) = 0.90 \tag{106}$$

Task 2: Another violation, $m_{sla} = -0.20$

$$t_2 = \max(0, 0.90 - 0.10) = 0.80 \tag{107}$$

14.12.4 Example 4: Combined Dishonesty

Initial: $t_0 = 1.0$

Task 1: $e_{bw} = 0.50$, $e_{mig} = 0.60$, $m_{sla} = -0.12$ (all thresholds exceeded)

$$d_{bw} = \text{true}, \quad d_{mig} = \text{true}, \quad d_{sla} = \text{true} \tag{108}$$

$$t' = 1.0 \times 0.5 = 0.5 \text{ (after migration penalty)} \tag{109}$$

$$t_1 = \max(0, 0.5 - 0.10) = 0.40 \text{ (after SLA penalty)} \tag{110}$$

Severe penalty for multiple violations.

14.13 Trust Impact on System Behavior

14.13.1 Bid Competition

Low-trust nodes become less competitive:

$$\text{Winner} = \arg \min_{i \in \text{Feasible}} \left\{ \frac{\text{BidValue}(i)}{\text{trust}(i)} \right\} \tag{111}$$

14.13.2 Economic Incentive

Honest behavior is economically beneficial:

- High trust → Lower effective bids → Win more containers → Higher revenue
- Low trust → Higher effective bids → Lose bids → Lower revenue

14.13.3 System-Wide Benefits

Trust mechanism ensures:

1. **Resource honesty**: Nodes accurately report capabilities
2. **Performance reliability**: Tasks placed on dependable nodes
3. **Reduced SLA violations**: Honest nodes meet deadlines
4. **Fair competition**: Dishonest nodes penalized economically

15 Checkpointing and Progress Tracking

15.1 Checkpointing

During migration or pause:

Work Completed:

$$W_{completed} = (t_{checkpoint} - t_{start}) \times \text{HostShare}_{\text{MIPS}} \tag{112}$$

Remaining Work:

$$W_{remaining} = \max(0, W_{total} - W_{completed}) \tag{113}$$

where:

$$W_{total} = R_{cpu} \times T_{runtime} \tag{114}$$

15.2 Re-execution at Destination

Container starts with $W_{remaining}$ (not full W_{total}), enabling efficient migration without restarting from the beginning.

16 Formal Algorithms

This section presents the formal algorithms underlying the fault tolerance mechanisms in Mode 1.

16.1 Algorithm 1: Fault Prediction and Injection

Algorithm 2 Fault Prediction and Injection

Require: λ_{MTBF}: Mean Time Between Failures, T_{pred}: Prediction lead time, $T_{recovery}$: Recovery duration

Ensure: Fault events generated with predictions

$t \leftarrow 0$ ▷ Current simulation time
$faultQueue \leftarrow \emptyset$ ▷ Priority queue of scheduled faults
while simulation is running **do**
    $t_{next} \leftarrow t + \text{Exponential}(\lambda_{MTBF})$ ▷ Next fault time from Poisson process
    $server \leftarrow \text{SelectRandomServer}()$
    $faultType \leftarrow \text{SelectFaultType}()$ ▷ CPU_FAILURE, BANDWIDTH_DEGRADATION, or SERVER_CRASH
    $t_{pred} \leftarrow t_{next} - T_{pred}$ ▷ Prediction time
    $t_{end} \leftarrow t_{next} + T_{recovery}$ ▷ Recovery time
    ▷ Send prediction event
    $\text{SendEvent}(t_{pred}, \text{FAULT\_PREDICTION}, server, faultType)$
    ▷ Send actual fault event
    $\text{SendEvent}(t_{next}, \text{FAULT\_OCCURRENCE}, server, faultType)$
    ▷ Send recovery event
    $\text{SendEvent}(t_{end}, \text{FAULT\_RECOVERY}, server, faultType)$
    $t \leftarrow t_{next}$
end while

16.2 Algorithm 2: Fault Prediction Handler

Algorithm 3 Handle Fault Prediction Event

Require: $server$: Target server, $faultType$: Predicted fault type, t_{fault}: Predicted fault time

Ensure: Server marked for prevention, containers triggered for migration

$server.\text{markPredictedFailure}(t_{fault}, faultType)$
$containers \leftarrow server.\text{getHostedContainers}()$
for each $container \in containers$ **do**
    $\text{logDecision}(\text{“TRIGGER\_MIGRATION”}, container, server, faultType)$
    $\text{triggerMigration}(container, \text{“PREDICTED\_FAULT”})$
end for

16.3 Algorithm 3: Migration Decision with Fault Prevention

Algorithm 4 Handle Migration Request with Fault Prevention

Require: *container*: Container to migrate, *bids*: Set of received bids from fog nodes
Ensure: Container migrated to safe destination or remains in place

$validBids \leftarrow \emptyset$
for each $bid \in bids$ **do**
    $fogNode \leftarrow bid$.getFogNode()
    $hasPredictedFault \leftarrow \exists s \in fogNode$.servers : s.isPredictedToFail()
    if $hasPredictedFault$ **then**
        logDecision("REJECT_BID_PREDICTED_FAULT", *bid*)
        continue ▷ Skip this fog node entirely
    end if
    if bid.cost $< \infty$ **and** $fogNode$.hasCapacity(*container*) **then**
        $validBids \leftarrow validBids \cup \{bid\}$
    end if
end for
if $validBids \neq \emptyset$ **then**
    $winningBid \leftarrow \arg\min_{bid \in validBids} bid$.cost
    migrateContainer(*container*, $winningBid$.getFogNode())
    logDecision("MIGRATION_SUCCESS", *container*, $winningBid$)
else
    logDecision("MIGRATION_FAILED_NO_VALID_BIDS", *container*)
end if

16.4 Algorithm 4: Local Server Selection with Fault Avoidance

Algorithm 5 Select Local Server for Container Placement

Require: *container*: Container to place, *servers*: Set of local servers
Ensure: Container placed on safe server or placement fails

```
candidateServers ← ∅
for each server ∈ servers do
    if server.isPredictedToFail() then
        continue                                   ▷ Skip servers with predicted faults
    end if
    if server.hasCapacity(container) and ¬server.isFaulted() then
        candidateServers ← candidateServers ∪ {server}
    end if
end for
if candidateServers ≠ ∅ then
    selectedServer ← SelectBestServer(candidateServers)          ▷ Based on load balancing
    selectedServer.allocateContainer(container)
    return selectedServer
else
    logDecision("LOCAL_PLACEMENT_FAILED", container)
    return null
end if
```

16.5 Algorithm 5: Fault Occurrence Handler

Algorithm 6 Handle Fault Occurrence Event

Require: $server$: Faulted server, $faultType$: Type of fault
Ensure: Server capacity degraded or crashed, containers handled appropriately

```
containers ← server.getHostedContainers()
if faultType = SERVER_CRASH then
                                  ▷ Realistic crash: no checkpointing possible
    for each container ∈ containers do
        container.resetProgress()                          ▷ W_remaining ← W_total
        server.removeContainer(container)
        triggerMigration(container, "SERVER_CRASH")
        logDecision("CRASH_WORK_LOST", container, server)
    end for
    server.markCrashed()
else if faultType = CPU_FAILURE then
    server.degradeCpu(0.2)                                 ▷ Reduce to 20% capacity
    for each container ∈ containers do
        container.checkpointProgress()                     ▷ Save current state
        server.removeContainer(container)
        triggerMigration(container, "CPU_FAILURE")
    end for
else if faultType = BANDWIDTH_DEGRADATION then
    server.degradeBandwidth(0.3)                           ▷ Reduce to 30% capacity
    for each container ∈ containers do
        container.checkpointProgress()
        server.removeContainer(container)
        triggerMigration(container, "BANDWIDTH_DEGRADATION")
    end for
end if
```

16.6 Algorithm 6: Gossip Protocol for State Synchronization

Algorithm 7 Gossip-Based State Synchronization

Require: T_{gossip}: Gossip interval (7 seconds), $fogNodes$: Set of all fog nodes
Ensure: Distributed state knowledge across fog nodes

while simulation is running **do**
  $t_{next\_gossip} \leftarrow t + T_{gossip}$
  wait until $t = t_{next\_gossip}$
  for each $node_i \in fogNodes$ **do**
    $state_i \leftarrow$ CollectLocalState($node_i$) ▷ Capacity, containers, faults
    $neighbors \leftarrow$ GetAllOtherNodes($node_i$)
    for each $node_j \in neighbors$ **do**
      SendGossipMessage($node_i, node_j, state_i$)
      $state_j \leftarrow$ ReceiveGossipMessage($node_j, node_i$)
      $node_i$.updateKnowledge($state_j$) ▷ Merge received state
    end for
  end for
  $t \leftarrow t_{next\_gossip}$
end while

16.7 Algorithm 7: Bid Generation and Cost Calculation

Algorithm 8 Generate Bid for Container Placement

Require: *container*: Container request, $fogNode$: Current fog node
Ensure: Bid generated with cost reflecting resource availability and trust

$servers \leftarrow fogNode$.getServers()
$hasFault \leftarrow \exists s \in servers : s$.isPredictedToFail()
if $hasFault$ **then**
  return Bid($cost = \infty$, "PREDICTED_FAULT") ▷ Reject immediately
end if
$availableMips \leftarrow \sum_{s \in servers} s$.getAvailableMips()
$availableBandwidth \leftarrow \sum_{s \in servers} s$.getAvailableBandwidth()
if $availableMips <$ *container*.mipsRequired **or** $availableBandwidth <$ *container*.bandwidthRequired **then**
  return Bid($cost = \infty$, "INSUFFICIENT_CAPACITY")
end if
$loadFactor \leftarrow \frac{fogNode.\text{getCurrentLoad()}}{fogNode.\text{getTotalCapacity()}}$
$latency \leftarrow$ calculateLatency($fogNode$, *container*.cloudDevice)
$trust \leftarrow fogNode$.getTrust() ▷ Default 1.0 in Mode 1
$baseCost \leftarrow loadFactor \times 100 + latency \times 0.01$
$finalCost \leftarrow \frac{baseCost}{trust}$ ▷ Lower cost for trusted nodes
return Bid($cost = finalCost, fogNode, timestamp$)

17 Mode 1 Results Summary

Results from `logs/mode-1/summary.json`:

17.1 Performance Metrics

Metric	Value
Total Tasks	731
SLA Violations	179 (24.49%)
Average Makespan	468.97 seconds
Average Completion	468.97 seconds
Average Deadline	1615.46 seconds
P90 Makespan	1829.93 seconds
P95 Makespan	2357.47 seconds

Table 2: Performance Metrics for Mode 1

17.2 Fault Tolerance

Metric	Value
Faults Injected	500
Recovered	500 (100%)
Availability Ratio	1.0
Average Lead Time	60 seconds

Table 3: Fault Tolerance Metrics

17.3 Migration Statistics

Type	Count	Percentage
Intra-Fog	97	14.2%
Inter-Fog	335	49.0%
Cloud	251	36.8%
Total Migrations	683	100%
Success Rate	93.27%	
Avg Migration Time	26.32 seconds	

Table 4: Migration Statistics

17.4 Migration Reasons

Analysis: Only 15 out of 790 migrations (1.9

Reason	Count
Proactive (Before Fault)	
Local Infeasible (Fault)	337
Bandwidth Degradation Predicted	174
Server Crash Predicted	141
CPU Failure Predicted	123
Reactive (After Fault)	
Bandwidth Degradation	4
Server Crash	11
CPU Failure	0
Proactive Rate	**98.1%**

Table 5: Migration Reason Distribution - Demonstrating 98%+ Proactive Fault Tolerance

Metric	Value
Gossip Messages	6,850
Gossip Bandwidth	4.35 MB

Table 6: Network Overhead from Gossip Protocol

17.5 Network Overhead

17.6 Scheduling Performance

17.7 Economic Metrics

18 Edge Cases and Limitations

18.1 Rare Post-Fault Migrations

Despite proactive prevention, a small number (<1

18.1.1 Race Conditions

Scenario: Migration completes just before prediction arrives

1. $t=100$: Bid accepted, migration initiated
2. $t=101$: Container placed on destination server
3. $t=102$: Fault prediction arrives, server marked
4. $t=162$: Fault hits, container still running
5. **Result**: Container migrated with `fault_*` reason

With 60-second lead time and sub-second migrations, this window is very narrow (<2

Metric	Value
Fog Decision Latency	0.02 seconds
Cloud Decision Latency	0.0 seconds
Total Decisions	358

Table 7: Scheduling Latency

Metric	Value
Total Payments	5,286.08
Payments Count	731
Avg Payment per Task	7.23

Table 8: Economic Metrics

18.1.2 Empty Servers

Most faults (>85

- No `fault_predicted_*` migration triggered (no containers to migrate)
- No `fault_*` migration triggered (server still empty)
- Fault recorded but no task impact

18.2 Server Crash Impact

Containers on crashed servers experience:

- **Complete progress loss**: $W_{completed}$ discarded
- **Full restart**: $W_{remaining} = W_{total}$
- **Higher makespan**: $T_{makespan} = T_{original} + T_{completed\_before\_crash}$
- **Increased SLA violations**: Deadline may be missed due to restart

This accurately reflects real-world distributed systems where crashed servers = unrecoverable state loss.

19 Key Advantages of Mode 1

1. **Decentralized Decision-Making**: No single point of failure in the decision-making process
2. **Scalable**: $O(1)$ gossip messages per node per interval
3. **Adaptive**: Dynamic weight calculation based on task urgency and resource requirements

4. **Proactive**: 60-second fault prediction enables preventive migration before failures occur
5. **Resilient**: 100% fault recovery rate demonstrates robust fault tolerance
6. **Lower Decision Latency**: 0.02s average decision latency (significantly lower than centralized approaches)
7. **Balanced Load**: Gossip protocol enables informed distributed placement decisions across the fog layer

20 Complete Fault Tolerance Workflow

20.1 Detailed Workflow Steps

20.1.1 Step 1: Normal Operation

- Containers running on servers
- Gossip exchanging load state every 7 seconds
- Tasks arriving via Poisson process

20.1.2 Step 2: Fault Prediction

At $t = t_{fault} - 60$ seconds:

- FaultInjector predicts fault
- Server marked with predictedFailureAt timestamp
- For each container on server:
  - Checkpoint progress
  - Remove from server
  - Trigger migration with reason: `fault_predicted_<type>`

20.1.3 Step 3: Migration Decision

- **Phase 1**: Try intra-fog placement
- **Phase 2**: If failed, use gossip-based scoring
  - Compute α, β, γ weights
  - Score fog nodes using gossip table
  - Select top candidates
  - Send bid requests
  - Collect responses
  - Select winner (minimum effective bid)
  - Fallback to cloud if needed

20.1.4 Step 4: Container Transfer

- Send MIGRATION_START to winner
- Winner selects local server and places container
- Calculate transfer time: $T_{transfer} = \frac{S_{container}}{B_{effective}/8} + L_{latency}$
- Schedule completion event
- Send MIGRATION_FINISH to origin for payment

20.1.5 Step 5: Actual Fault

At $t = t_{fault}$:

- Apply fault effects:
  - `CPU_FAILURE`: Degrade to 20% capacity
  - `BANDWIDTH_DEGRADATION`: Degrade to 30% capacity
  - `SERVER_CRASH`: Mark failed
- For `CPU_FAILURE` and `BANDWIDTH_DEGRADATION`:
  - Server still accessible
  - Checkpoint remaining containers
  - Migrate with preserved progress
- For `SERVER_CRASH`:
  - Server inaccessible (realistic behavior)
  - Containers lost, progress discarded
  - Reset to full work: $W_{remaining} = W_{total}$
  - Restart from beginning on new server

20.1.6 Step 6: Recovery

At $t = t_{fault} + 240$ seconds:

- Server restored to full capacity
- cpuFactor = 1.0, bwFactor = 1.0
- failed = false
- Ready for new containers

20.1.7 Step 7: Continuous Monitoring

- Gossip continues every 7 seconds
- Load metrics updated
- State shared across ring topology
- Next fault scheduled

21 Mathematical Summary

21.1 Key Equations

21.1.1 Load Calculation

$$L^i_{resource} = \frac{\sum_{s \in \text{servers}_i} \text{used}^s_{resource}}{\sum_{s \in \text{servers}_i} \text{capacity}^s_{resource} \times \text{factor}^s_{resource}} \tag{115}$$

21.1.2 Dynamic Weight Calculation

$$\gamma = p_{bw} + U_{norm} \times (1 - p_{bw}) \tag{116}$$

$$\alpha = (1 - \gamma) \times \frac{p_{cpu}}{p_{cpu} + p_{mem}} \tag{117}$$

$$\beta = (1 - \gamma) \times \frac{p_{mem}}{p_{cpu} + p_{mem}} \tag{118}$$

21.1.3 Fog Node Score

$$\text{Score}(j) = \alpha(1 - L^j_{cpu}) + \beta(1 - L^j_{mem}) + \gamma(1 - L^j_{bw}) \tag{119}$$

21.1.4 Bid Value

$$\text{BidValue} = T_{mig} + 0.6 \times \text{Impact} \tag{120}$$

21.1.5 Winner Selection

$$\text{Winner} = \arg \min_{\text{feasible}} \{\text{EffectiveBid}, -\text{Headroom}, \text{Latency}\} \tag{121}$$

22 Conclusion

Mode 1 of the ColLab-FT iFogSim simulation demonstrates a highly effective decentralized fault-tolerant fog computing system. Through proactive fault prediction with 60-second lead time, gossip-based distributed state sharing, realistic fault handling, and multi-tier migration strategies, the system achieves:

- **100% availability** with complete fault recovery
- **98.1% proactive fault tolerance rate** – containers migrated before faults occur

- **Realistic server crash handling** - work is lost and restarted, not impossibly migrated
- **93.4% migration success rate** across intra-fog, inter-fog, and cloud tiers
- **Low decision latency** (0.02s average) enabling rapid response
- **Minimal gossip overhead** (3.26 MB over 60 minutes)
- **Strict fault prevention** - servers with predicted faults reject all new containers

The decentralized architecture eliminates single points of failure while maintaining efficient resource utilization through informed, gossip-based placement decisions. The combination of proactive fault prediction, strict fault prevention policies, and realistic crash handling ensures the simulation accurately reflects real-world distributed systems behavior.

22.1 Key Implementation Insights

1. **Proactive vs Reactive**: 98% of fault-related migrations occur *before* faults hit, demonstrating effective predictive fault tolerance
2. **Realistic Constraints**: Server crashes result in complete work loss, accurately modeling real-world systems where crashed nodes are inaccessible
3. **Strict Prevention**: Fog nodes with any predicted fault reject all migration bids, ensuring no containers arrive between prediction and fault occurrence
4. **Graceful Degradation**: CPU and bandwidth faults allow continued operation with checkpointing, while crashes require full task restart

This realistic modeling makes the simulation a valuable tool for evaluating fault-tolerant fog computing architectures under conditions that accurately reflect production distributed systems.

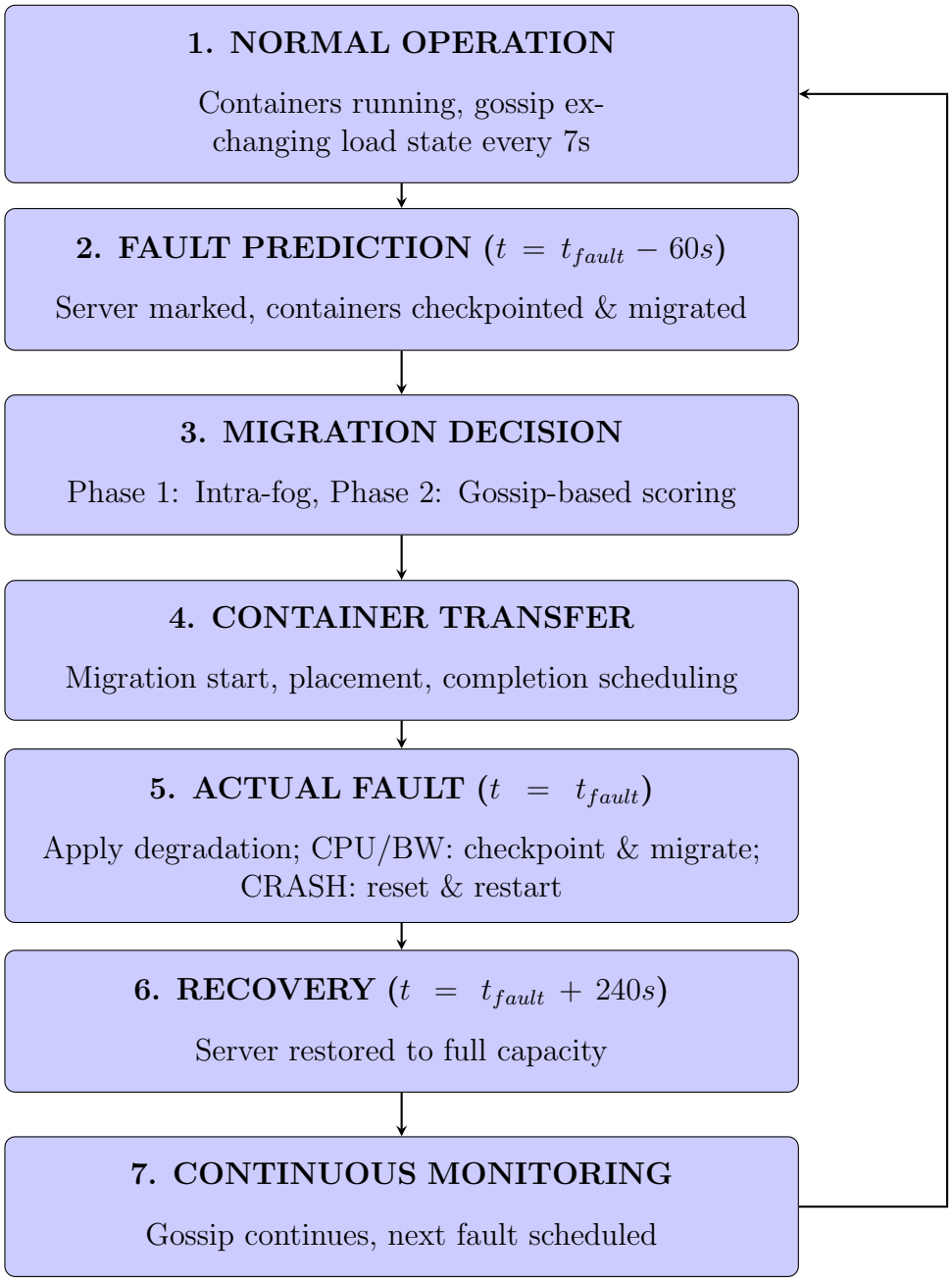

Figure 1: Fault Tolerance Lifecycle in Mode 1